

Notebook 01 — Exploratory Data Analysis

Project: Ad Engagement Analysis

Goal: Understand campaign interaction patterns

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier, AdaBoostClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import classification_report, accuracy_score, roc_auc_score, f1_score
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier

# Set style for seaborn
sns.set_theme(style="whitegrid")
import os
import sys
import warnings
warnings.filterwarnings('ignore')

# Load the provided Kaggle data directly
try:
    df = pd.read_csv('../data/comments_cleaned.csv')
    print("Dataset 'comments_cleaned.csv' loaded successfully.")
```

```

    print("Shape:", df.shape)
except FileNotFoundError:
    print("Dataset not found. Please ensure 'comments_cleaned.csv' is in the 'data' folder.")
    sys.exit(1)

display(df.head())

```

Dataset 'comments_cleaned.csv' loaded successfully.
Shape: (7488, 9)

	Unnamed: 0	id	comment	User id	Photo id	created Timestamp	posted date	emoji used	Hashtags used count
0	0	1	unde at dolore	2	1	13-04-2023 08:04	April 14	yes	1
1	1	2	quae ea ducimus	3	1	13-04-2023 08:04	April 14	no	2
2	2	3	alias a voluptatum	5	1	13-04-2023 08:04	April 14	no	4
3	3	4	facere suscipit sunt	14	1	13-04-2023 08:04	April 14	yes	2
4	4	5	totam eligendi quaerat	17	1	13-04-2023 08:04	April 14	yes	1

1. Environment Setup and Data Loading

Import essential data science libraries (Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn) and load the Instagram user behavior dataset.

2. Descriptive Statistics and EDA

Let's look at the basic distributions of our core user engagement metrics.

```

In [2]: # Basic info
print(df.info())

# Remove unnamed index column if it exists
if 'Unnamed: 0' in df.columns:
    df.drop(columns=['Unnamed: 0'], inplace=True)
elif 'id' in df.columns and len(df['id'].unique()) == len(df):
    df.drop(columns=['id'], inplace=True)

```

```
# Feature Engineering
# Let's derive comment length to capture 'effort' of engagement
df['comment_length'] = df['comment'].astype(str).str.len()

# Convert Timestamp to Datetime and extract components
df['created Timestamp'] = pd.to_datetime(df['created Timestamp'], errors='coerce')
df['hour_posted'] = df['created Timestamp'].dt.hour
df['day_of_week'] = df['created Timestamp'].dt.dayofweek
# Fillna with mode for dates that didn't parse correctly
df['hour_posted'].fillna(df['hour_posted'].mode()[0], inplace=True)
df['day_of_week'].fillna(df['day_of_week'].mode()[0], inplace=True)

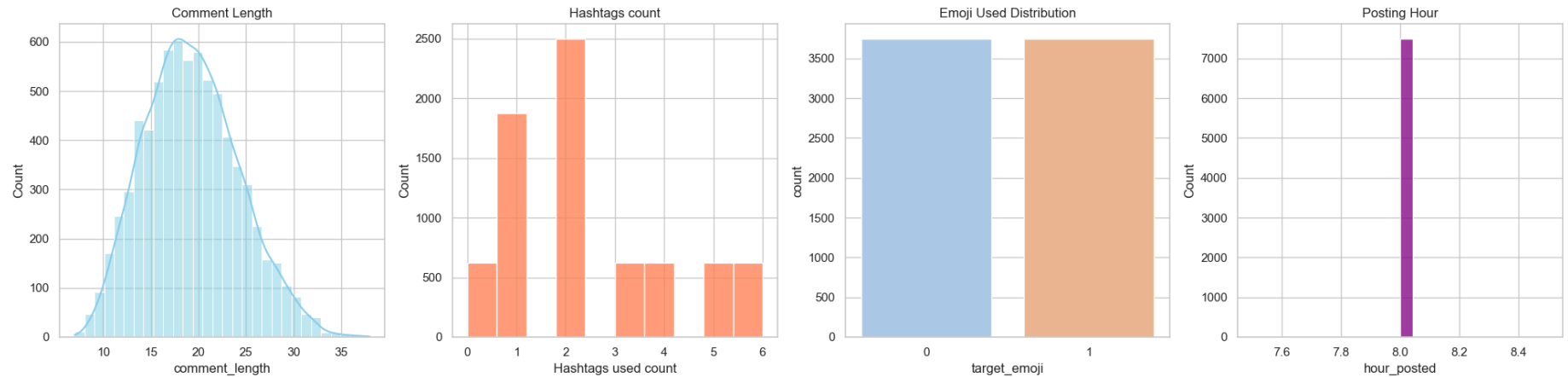
# Target processing
# Convert 'emoji used' from 'yes'/'no' string to binary target
df['target_emoji'] = df['emoji used'].map({'yes': 1, 'no': 0})
# Map just in case there are missing target values
df.dropna(subset=['target_emoji'], inplace=True)

print("Target Distribution:")
print(df['target_emoji'].value_counts())

# Check for missing values
print(f"Missing Values:\n{df.isnull().sum()}")

# Visualizing distributions
fig, axes = plt.subplots(1, 4, figsize=(20, 5))
sns.histplot(df['comment_length'], bins=30, kde=True, ax=axes[0], color='skyblue').set_title('Comment Length')
sns.histplot(df['Hashtags used count'], bins=10, kde=False, ax=axes[1], color='coral').set_title('Hashtags count')
sns.countplot(x='target_emoji', data=df, ax=axes[2], palette='pastel').set_title('Emoji Used Distribution')
sns.histplot(df['hour_posted'], bins=24, ax=axes[3], color='purple').set_title('Posting Hour')
plt.tight_layout()
plt.show()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7488 entries, 0 to 7487
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Unnamed: 0            7488 non-null  int64
1   id                    7488 non-null  int64
2   comment              7488 non-null  object
3   User id              7488 non-null  int64
4   Photo id             7488 non-null  int64
5   created Timestamp    7488 non-null  object
6   posted date          7488 non-null  object
7   emoji used           7488 non-null  object
8   Hashtags used count  7488 non-null  int64
dtypes: int64(5), object(4)
memory usage: 526.6+ KB
None
Target Distribution:
target_emoji
1    3744
0    3744
Name: count, dtype: int64
Missing Values:
id                0
comment           0
User id           0
Photo id          0
created Timestamp 0
posted date       0
emoji used        0
Hashtags used count 0
comment_length    0
hour_posted       0
day_of_week       0
target_emoji      0
dtype: int64
```



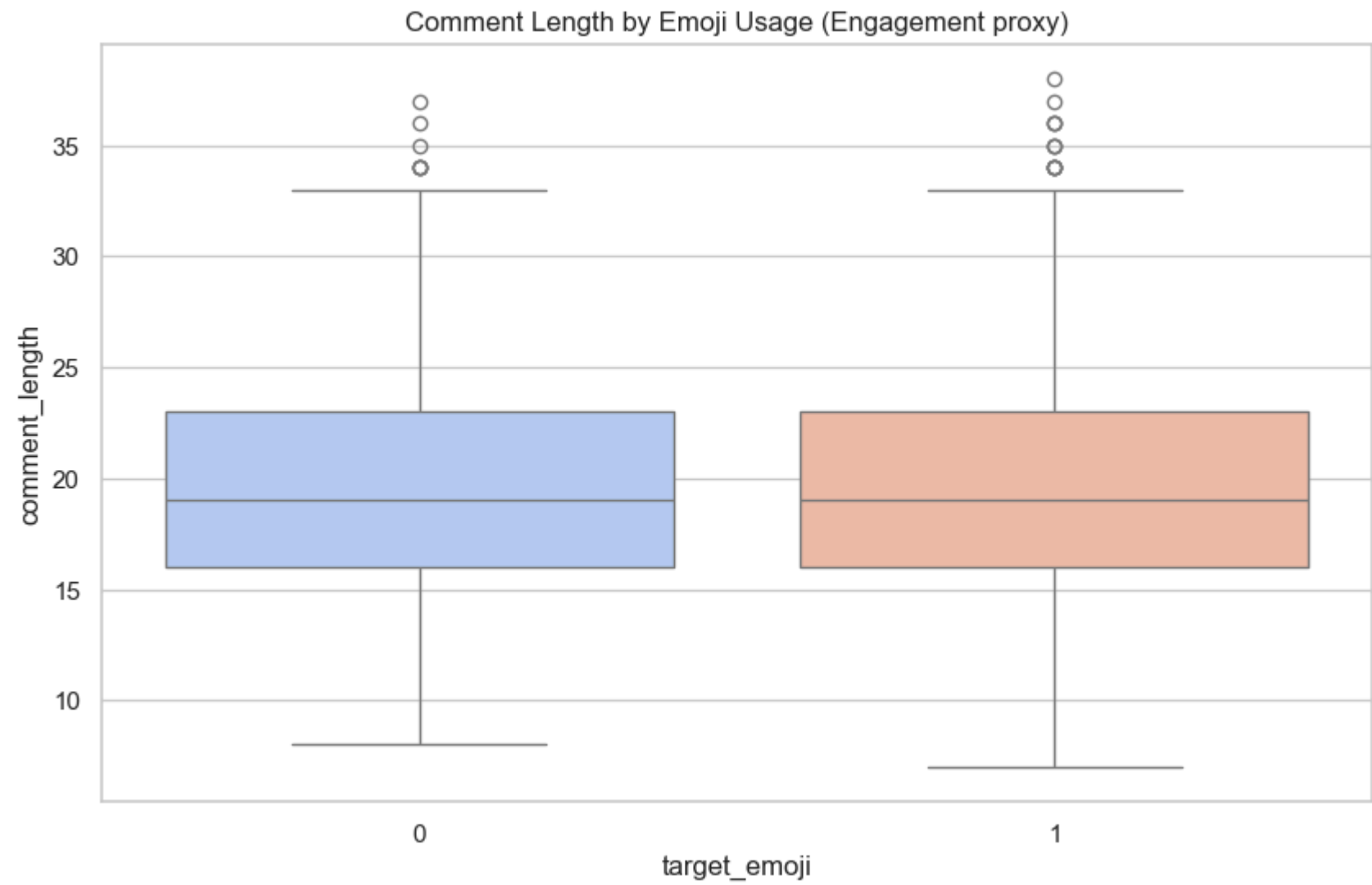
3. Exploratory Data Analysis: User Ad Interactions

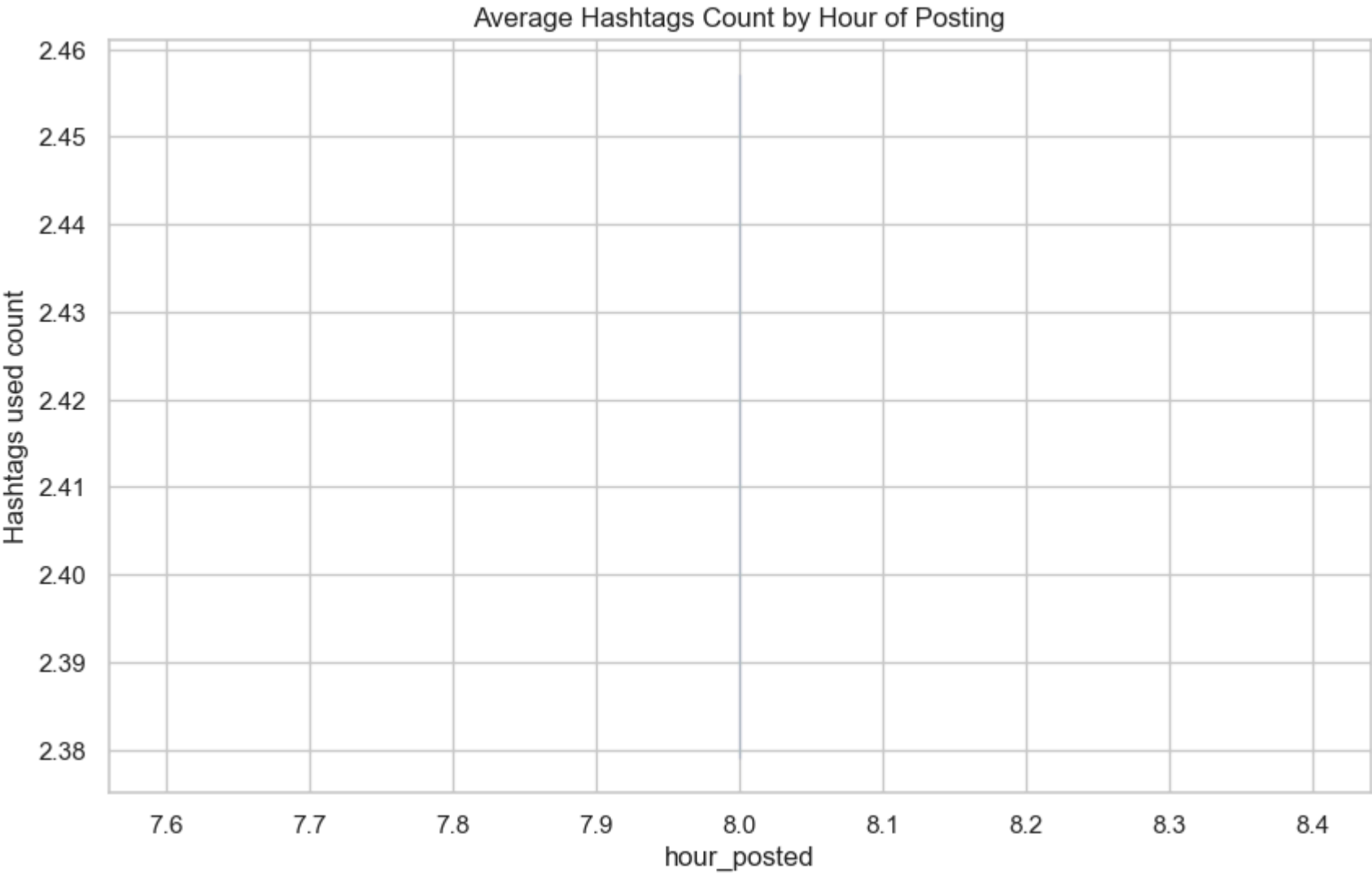
Let's see the relationships between different interaction metrics, time spent, and ad clicks. Calculate CTR.

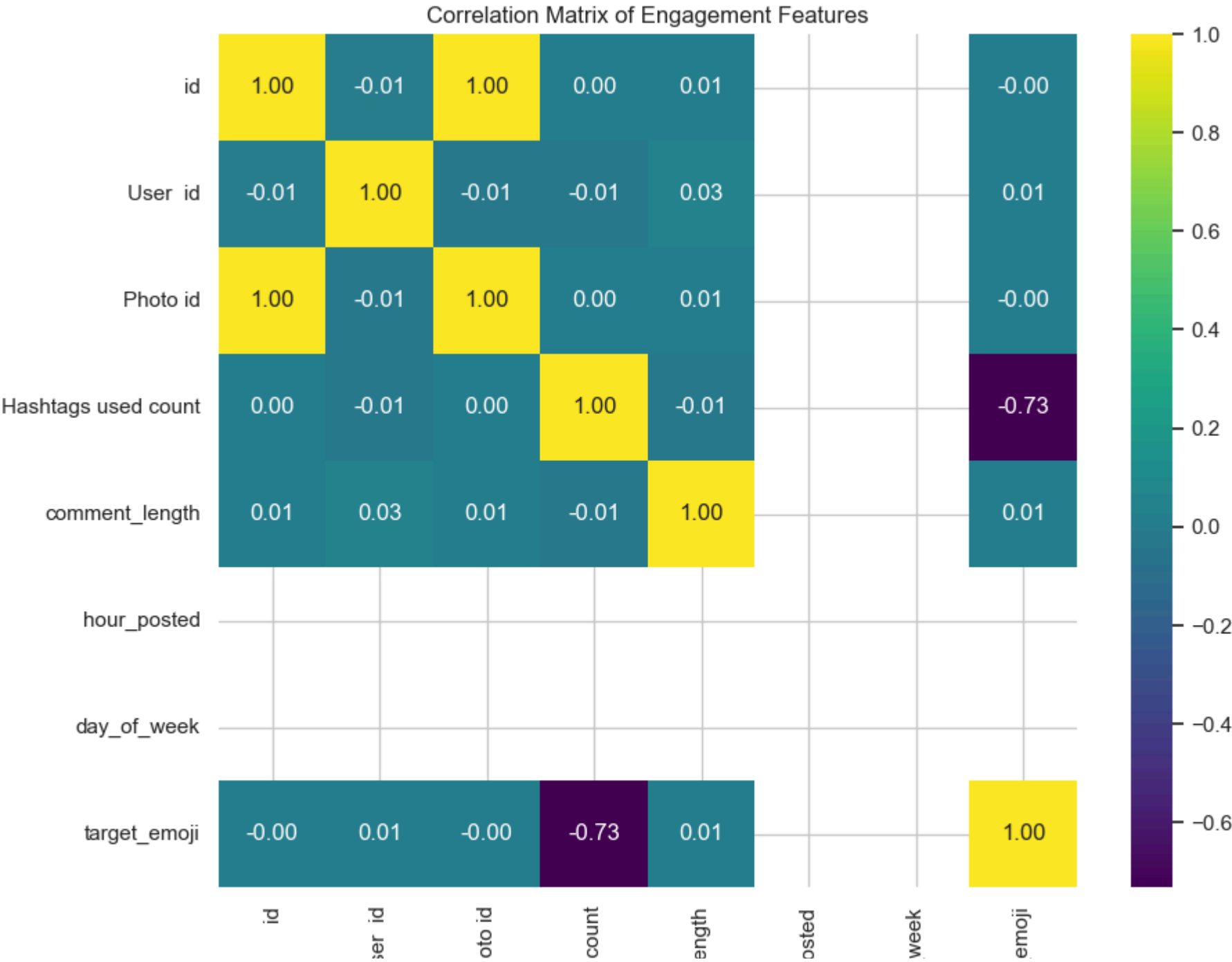
```
In [3]: # Focus on how features interact with the decision to use emojis or hashtags
plt.figure(figsize=(10,6))
sns.boxplot(x='target_emoji', y='comment_length', data=df, palette='coolwarm')
plt.title('Comment Length by Emoji Usage (Engagement proxy)')
plt.show()

# Let's see the hour vs hashtag counts
plt.figure(figsize=(10,6))
sns.lineplot(x='hour_posted', y='Hashtags used count', data=df)
plt.title('Average Hashtags Count by Hour of Posting')
plt.show()

# Calculate Correlation Matrix to identify linear relationships
plt.figure(figsize=(10,8))
corr = df.select_dtypes(include=[np.number]).corr()
sns.heatmap(corr, annot=True, cmap='viridis', fmt='.2f')
plt.title('Correlation Matrix of Engagement Features')
plt.show()
```







Us
Ph
Hashtags used
comment_l
hour_pi
day_of_
target_

4. Statistical Analysis of Campaign Effectiveness

Hypothesis Testing: Independent samples T-test to measure if engagement differs significantly by gender.

```
In [4]: from scipy import stats

print("Hypothesis Testing: Do users who use more hashtags leave longer comments?")
# Let's split into 'few hashtags' vs 'many hashtags'
median_hashtags = df['Hashtags used count'].median()
group_high = df[df['Hashtags used count'] > median_hashtags]['comment_length']
group_low = df[df['Hashtags used count'] <= median_hashtags]['comment_length']

# Two-sample t-test
t_stat, p_val = stats.ttest_ind(group_high.dropna(), group_low.dropna(), equal_var=False)

print(f"\nIndependent T-test Results:")
print(f"T-statistic: {t_stat:.4f}")
print(f"P-value: {p_val:.4f}")

if p_val < 0.05:
    print("Conclusion: Reject null hypothesis. There is a statistically significant difference in comment length between users who use more hashtags and those who use fewer hashtags.")
else:
    print("Conclusion: Fail to reject null hypothesis. No statistically significant difference found.")
```

Hypothesis Testing: Do users who use more hashtags leave longer comments?

Independent T-test Results:

T-statistic: -1.2093

P-value: 0.2266

Conclusion: Fail to reject null hypothesis. No statistically significant difference found.

5. Predictive Modeling for Ad Conversion

Use machine learning classifiers (Logistic Regression and Random Forest) to predict whether a user will convert based on their historical behavior metrics.

```
In [5]: # Data Prep
# Target: Will the user employ emojis? (An indicator of qualitative ad engagement)
# Drop raw text, parsed dates, and identifiers which won't model simply for binary classification
features_to_drop = ['Unnamed: 0', 'id', 'comment', 'User id', 'Photo id', 'created Timestamp', 'posted date', 'emo
missing_drop = [c for c in features_to_drop if c in df.columns]

X = df.drop(columns=missing_drop).dropna()
y = df.loc[X.index, 'target_emoji']

print("Features selected for modeling:", X.columns.tolist())

# Train Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Feature Scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# To go full scale, let's establish a dictionary of several models to test simultaneously!
models = {
    "Logistic Regression": LogisticRegression(random_state=42),
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
    "Gradient Boosting": GradientBoostingClassifier(random_state=42),
    "AdaBoost": AdaBoostClassifier(random_state=42),
    "Support Vector Machine": SVC(probability=True, random_state=42),
    "K-Nearest Neighbors": KNeighborsClassifier(n_neighbors=5),
    "XGBoost": XGBClassifier(use_label_encoder=False, eval_metric='logloss', random_state=42),
    "LightGBM": LGBMClassifier(random_state=42),
    "Neural Network (MLP)": MLPClassifier(hidden_layer_sizes=(100, 50), max_iter=500, random_state=42)
}

# Results dictionary to store evaluation metrics
```

```
results = {}

# Fit all models
for name, model in models.items():
    print(f"Training {name}...")
    try:
        # Scale input automatically if model typically benefits from it
        if name in ["Logistic Regression", "Support Vector Machine", "K-Nearest Neighbors", "Neural Network (MLP)"]:
            model.fit(X_train_scaled, y_train)
            y_pred = model.predict(X_test_scaled)
            y_prob = model.predict_proba(X_test_scaled)[:, 1] if hasattr(model, "predict_proba") else None
        else:
            model.fit(X_train, y_train)
            y_pred = model.predict(X_test)
            y_prob = model.predict_proba(X_test)[:, 1] if hasattr(model, "predict_proba") else None

        acc = accuracy_score(y_test, y_pred)
        auc = roc_auc_score(y_test, y_prob) if y_prob is not None else "N/A"
        f1 = f1_score(y_test, y_pred)
        results[name] = {"Accuracy": acc, "ROC-AUC": auc, "F1-Score": f1, "Model": model}
    except Exception as e:
        print(f"Failed configuring {name}: {e}")

print("\nModel Training Cycle Completed!")
```

Features selected for modeling: ['Hashtags used count', 'comment_length', 'hour_posted', 'day_of_week']
Training Logistic Regression...
Training Random Forest...
Training Gradient Boosting...
Training AdaBoost...
Training Support Vector Machine...

[illegible]

[illegible]


```
best_auc = 0
best_model = None

print(f"{'Model Profile':<25} | {'Accuracy':<10} | {'ROC-AUC':<10} | {'F1-Score':<10}")
print("-" * 65)

for name, metrics in results.items():
    auc_val = metrics["ROC-AUC"] if isinstance(metrics["ROC-AUC"], float) else 0.0
    print(f"{name:<25} | {metrics['Accuracy']:<10.4f} | {auc_val:<10.4f} | {metrics['F1-Score']:<10.4f}")

    if auc_val > best_auc:
        best_auc = auc_val
        best_model_name = name
        best_model = metrics["Model"]

print(f"\n--- Best Performing Model Selected ---")
print(f"🏆 The winner goes to: {best_model_name} with an AUC of {best_auc:.4f}")

# Extracting Feature Importance from the best tree-based model (if applicable)
if hasattr(best_model, "feature_importances_"):
    importances = best_model.feature_importances_
    feat_imp_df = pd.DataFrame({'Feature': X.columns, 'Importance': importances}).sort_values(by='Importance', ascending=False)

    plt.figure(figsize=(10,6))
    sns.barplot(x='Importance', y='Feature', data=feat_imp_df, palette='magma')
    plt.title(f'Feature Importances - {best_model_name}')
    plt.xlabel('Importance Score')
    plt.ylabel('Features')
    plt.show()
else:
    print(f"{best_model_name} lacks straightforward `.feature_importances_` to plot.")
```

Model Profile	Accuracy	ROC-AUC	F1-Score
Logistic Regression	0.8304	0.9434	0.8472
Random Forest	0.8284	0.9445	0.8274
Gradient Boosting	0.8371	0.9461	0.8336
AdaBoost	0.8304	0.9427	0.8539
Support Vector Machine	0.8364	0.9449	0.8323
K-Nearest Neighbors	0.8378	0.9446	0.8350
XGBoost	0.8284	0.9450	0.8272
LightGBM	0.8284	0.9450	0.8272
Neural Network (MLP)	0.8338	0.9449	0.8341

--- Best Performing Model Selected ---

🏆 The winner goes to: Gradient Boosting with an AUC of 0.9461

